

# KCF Object Tracking

---

## KCF Object Tracking

1. Program Functionality
2. Program Code Reference Path
3. Program Startup
  - 3.1 Startup Command
  - 3.2 Dynamic Parameter Adjustment
4. Core Code
  - 4.1. `mono_Tracker.py`

## 1. Program Functionality

---

After launching the program, use your mouse to select the object you want to track. Press the spacebar to enter tracking mode. The servo gimbal will lock onto the selected object, keeping it in the center of the screen. Press the q/Q key to exit the program.

⚠ This function works best with larger selections. Move slowly to avoid losing the target.

## 2. Program Code Reference Path

---

```
~/yahboomcar_ros2_ws/yahboomcar_ws/src/yahboomcar_astra/yahboomcar_astra/mono_Tracker.py
~/yahboomcar_ros2_ws/yahboomcar_ws/src/yahboomcar_astra/yahboomcar_astra/common/track_common.py
```

- `mono_Tracker.py`  
Mainly performs object detection and tracking. Based on the detected object center coordinates, it calculates the desired servo rotation angle and sends the servo angle control data to the robot.
- `track_common.py`
  - Implements multiple OpenCV tracking algorithms (BOOSTING, KCF, CSRT, etc.)
  - Provides tracker initialization (`initWorking`) and real-time update (`track`) interfaces

## 3. Program Startup

---

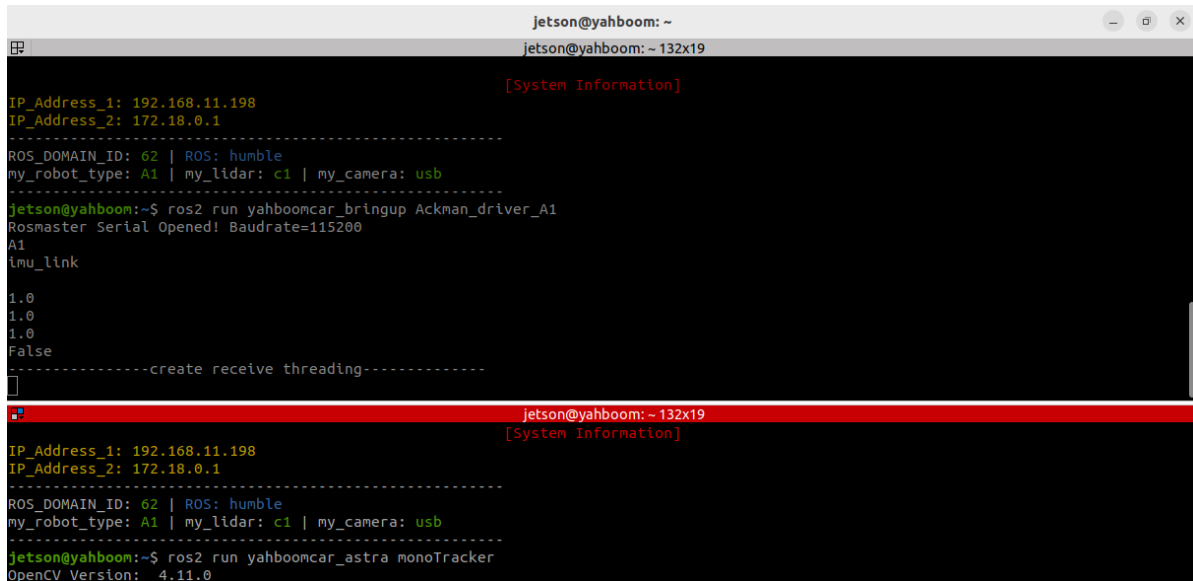
### 3.1 Startup Command

**For PI5/JETSON NANO controllers, you need to enter a Docker container first; for RDK X5 and Orin boards, this is not required.**

**Enter the Docker container (see the Docker section [3. Docker Commit and Multi-Terminal Entry] for steps).**

Enter the following in the terminal:

```
# Start the car chassis
ros2 run yahboomcar_bringup Ackman_driver_A1
# Start the KCF object tracking program
ros2 run yahboomcar_astra monoTracker
```



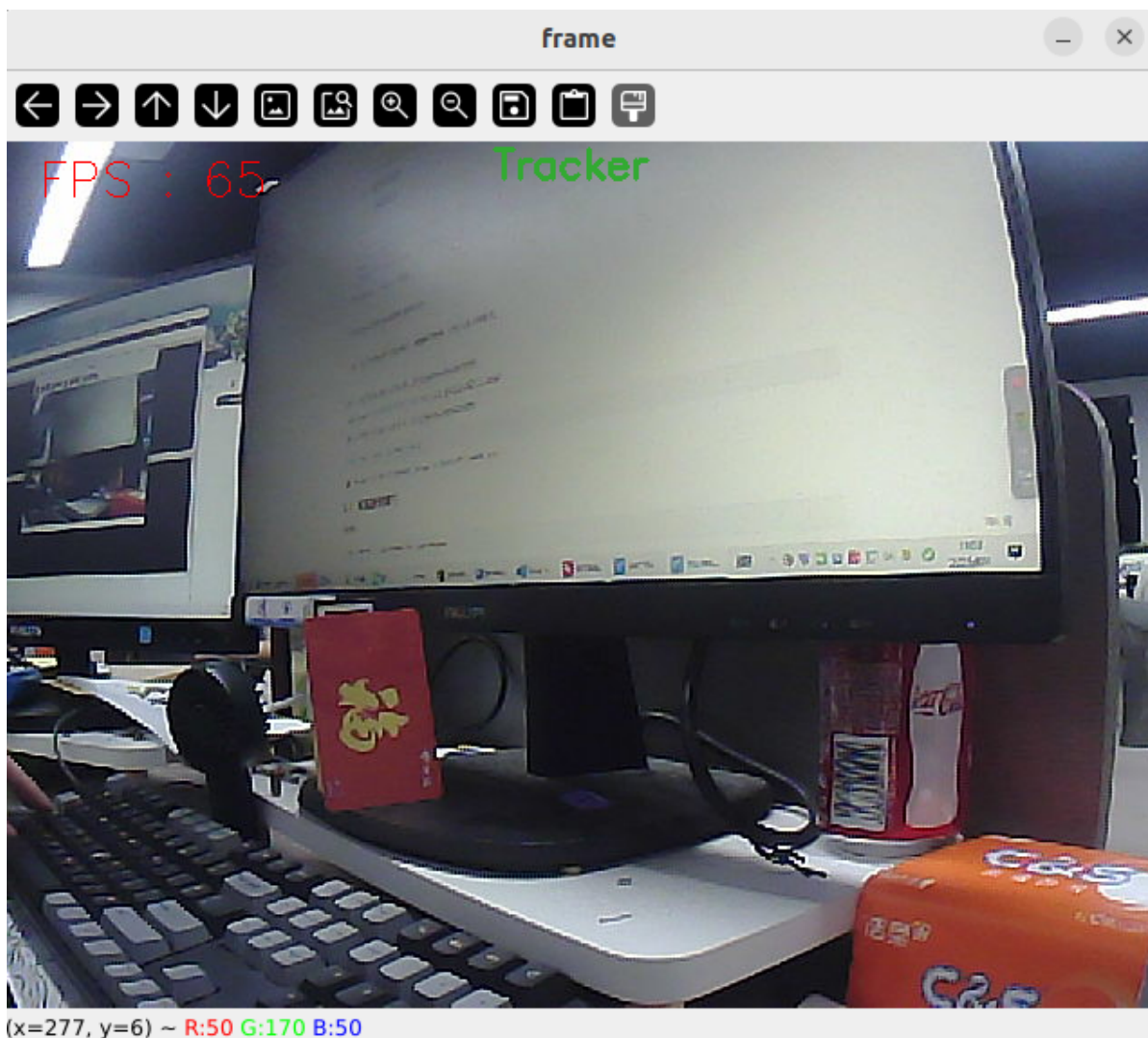
The image shows two terminal windows from a Jetson Nano. The top window displays system information for ROS2 Humble, including IP addresses (192.168.11.198 and 172.18.0.1), domain ID (62), robot type (A1), and sensor configurations (lidar: c1, camera: usb). It shows the execution of `ros2 run yahboomcar_bringup Ackman_driver_A1`, which successfully opens the Rosmaster serial connection at 115200 baudrate and starts the Ackman driver. The bottom window shows the execution of `ros2 run yahboomcar_astra monoTracker`, which starts the OpenCV 4.11.0-based tracking application.

```
jetson@yahboom: ~
jetson@yahboom: ~ 132x19

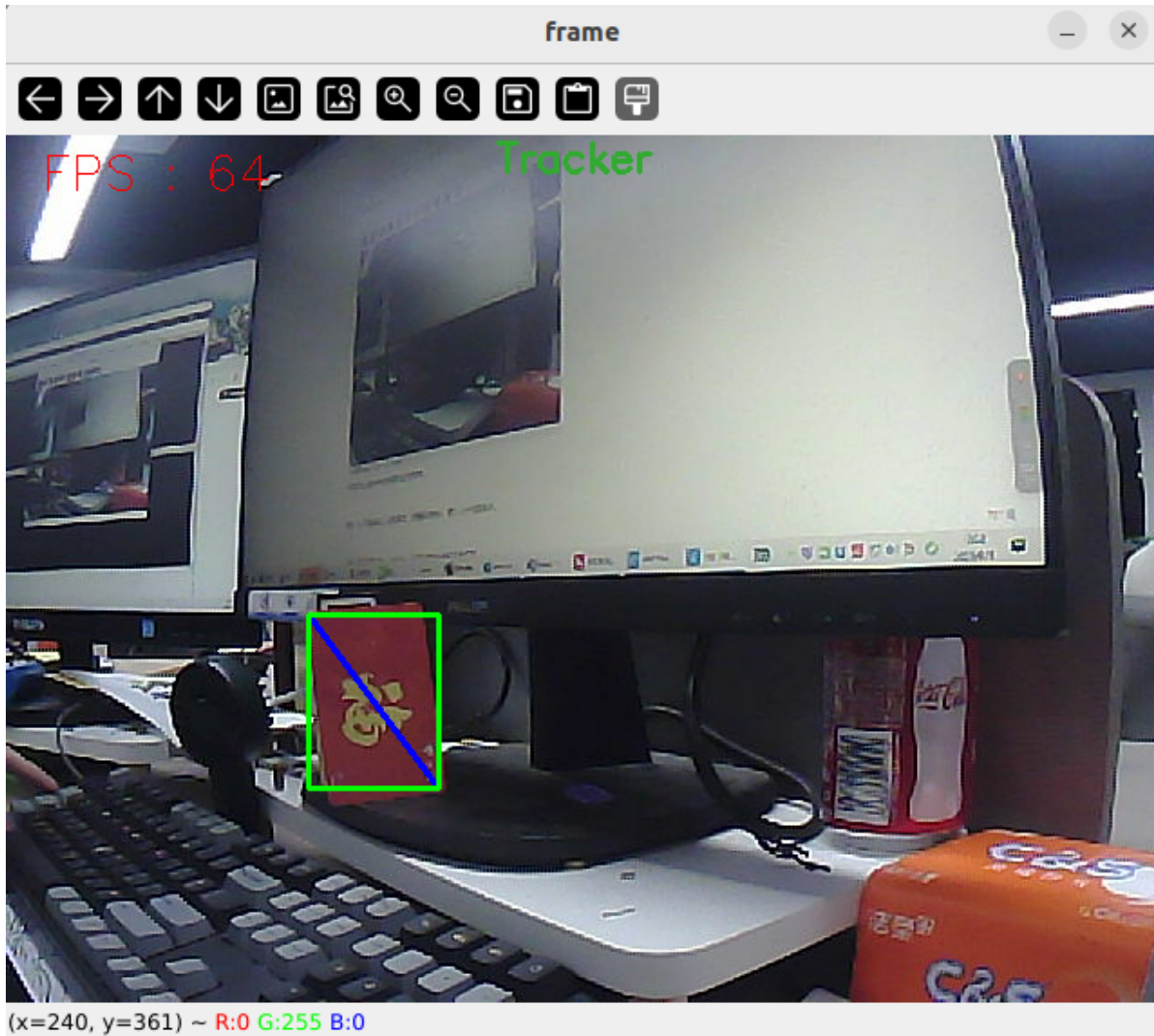
[System Information]
IP_Address_1: 192.168.11.198
IP_Address_2: 172.18.0.1
-----
ROS_DOMAIN_ID: 62 | ROS: humble
my_robot_type: A1 | my_lidar: c1 | my_camera: usb
-----
jetson@yahboom:~$ ros2 run yahboomcar_bringup Ackman_driver_A1
Rosmaster Serial Opened! Baudrate=115200
A1
imu_link
1.0
1.0
1.0
False
-----create receive threading-----
[]

jetson@yahboom: ~ 132x19
[System Information]
IP_Address_1: 192.168.11.198
IP_Address_2: 172.18.0.1
-----
ROS_DOMAIN_ID: 62 | ROS: humble
my_robot_type: A1 | my_lidar: c1 | my_camera: usb
-----
jetson@yahboom:~$ ros2 run yahboomcar_astra monoTracker
OpenCV Version: 4.11.0
```

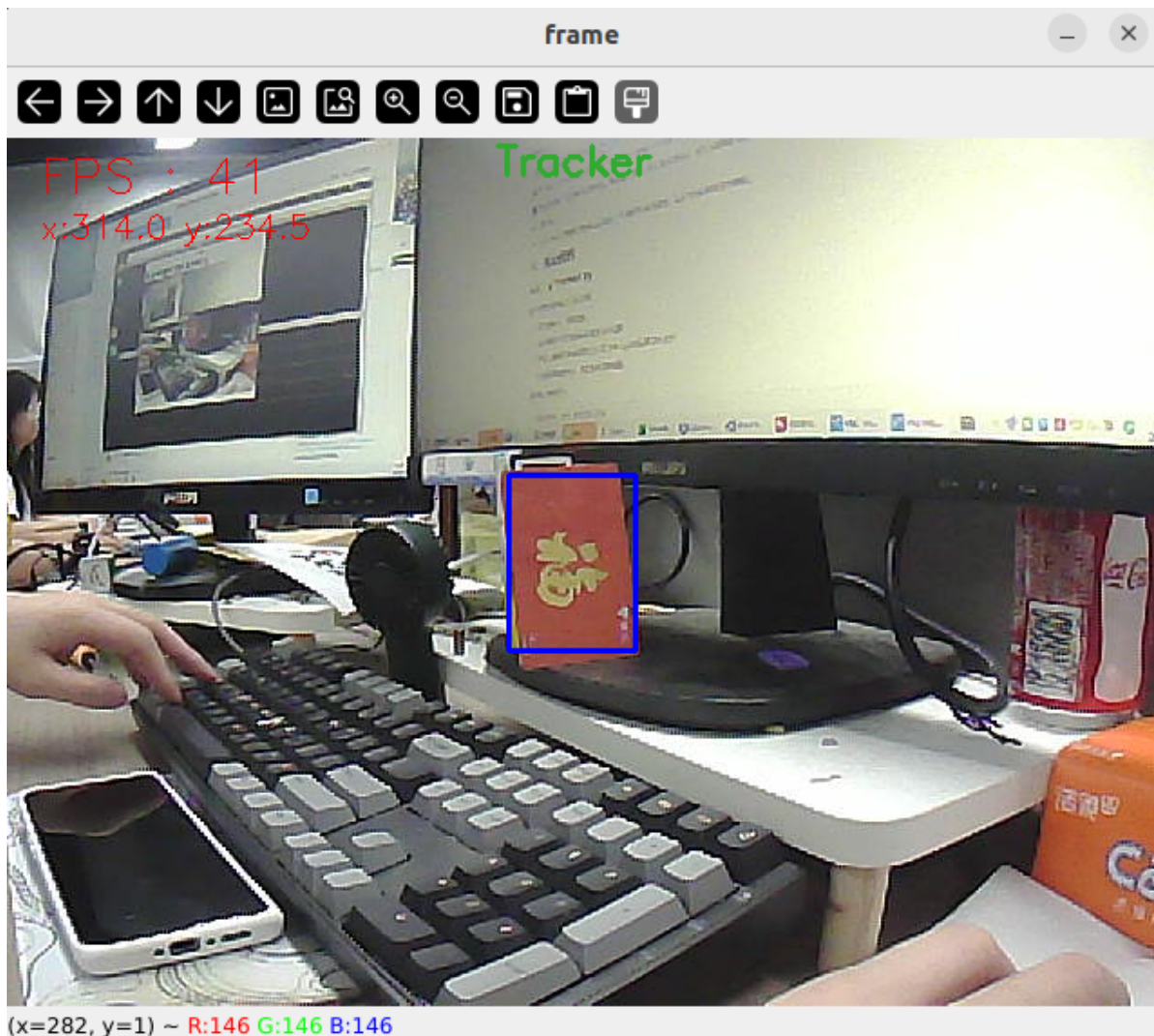
After the program starts, the following screen will appear.



Use your mouse to select the object you want to track.



Then, press the spacebar to enter tracking mode. Slowly move the object, and the servo gimbal will follow.



In addition, we can enter the following command to print information about the target center coordinates:

```
ros2 topic echo /Current_point
```

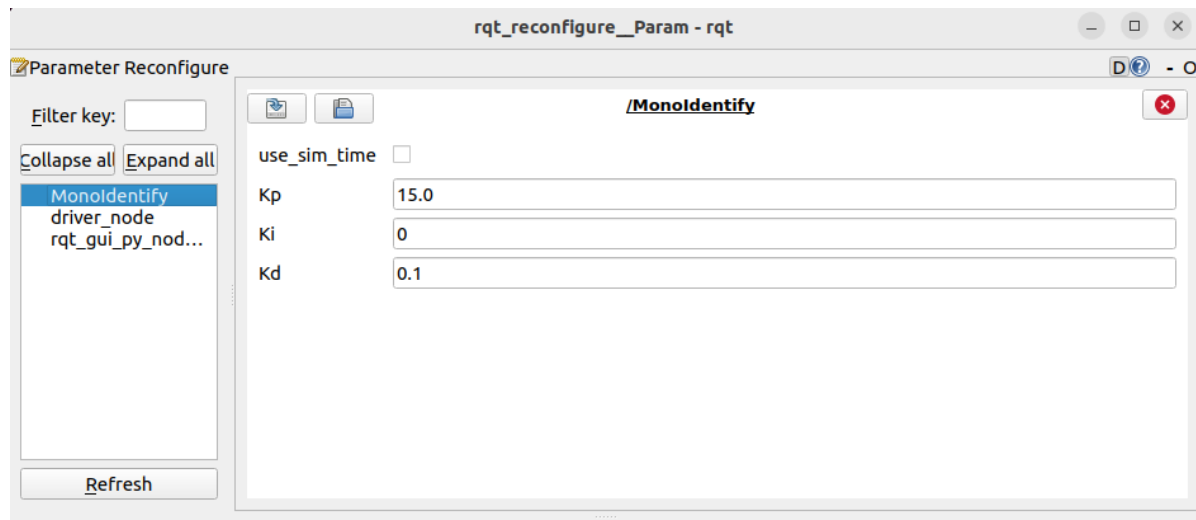
```
jetson@yahboom:~$ ros2 topic echo /Current_point
anglex: 42.0
angley: 220.0
distance: 0.0
---
anglex: 49.0
angley: 222.0
distance: 0.0
---
```

## 3.2 Dynamic Parameter Adjustment

In the terminal, enter:

```
ros2 run rqt_reconfigure rqt_reconfigure
```





(System message might be shown here when necessary)

☑ After modifying the parameters, click a blank area in the GUI to write the parameter values. Note that the values will only take effect during the current boot. To permanently apply them, you need to modify the parameters in the source code.

As shown in the figure above,

- mono\_Tracker is primarily responsible for servo gimbal movement, adjusting PID-related parameters to achieve optimal gimbal motion.

✂ Parameter Analysis:

[Kp], [Ki], [Kd]: PID control of the servo gimbal speed during tracking.

## 4. Core Code

### 4.1. mono\_Tracker.py

This program has the following main functions:

- Initialize the KCF tracker
- Open the camera and acquire an image
- Select the tracking target using mouse interaction
- Calculate the target center coordinates and publish them
- Use the PID algorithm to calculate the servo angle and issue control commands

Some core code is as follows:

```
# Create a publisher to publish the center coordinates of the tracked object
self.pub_position = self.create_publisher(Position, "/Current_point", 10)
# Define the servo data publisher
self.pub_Servo = self.create_publisher(ServoControl, 'Servo', 10)
...
# Tracker initialization
self.tracker_types = ['BOOSTING', 'MIL', 'KCF']
self.tracker_type = ['KCF']
self.gTracker = Tracker(tracker_type=self.tracker_type)
...
# Mouse callback function, used to select the tracking target
def onMouse(self, event, x, y, flags, param):
    if event == 1: # Left mouse button pressed
        self.Track_state = 'init'
```

```

        self.select_flags = True
        self.Mouse_XY = (x,y)
    if event == 4: # Left mouse button released
        self.select_flags = False
        self.Track_state = 'identify'
    ...
# Calculate the servo angle using the PID algorithm based on the x and y values
def execute(self, point_x, point_y):
    position = Position()
    position.angle_x = point_x * 1.0
    position.angle_y = point_y * 1.0
    # Publish the center coordinates message
    self.pub_position.publish(position)
    ...
    # Limit PID polarity and maximum value
    [x_Pid, y_Pid] = self.PID_controller.update([point_x - 320, point_y - 240])
    x_Pid = x_Pid * (abs(x_Pid) <= self.Kp/3)
    y_Pid = y_Pid * (abs(y_Pid) <= self.Kp/3)
    ...
    # Publish the calculated servo angle
    self.pub_Servo.publish(self.servo_angle)

```